



Real-Time Anomaly Detection with Contextual Log Parsing in Distributed Systems

SRDS 2026

ERC 2026-2

Hadjou Ilyas | Dr. Irshad Khan | Prof. Young-Woo Kwon

Presented by Hadjou Ilyas

1

INTRODUCTION

2

MOTIVATION

3

PROPOSED APPROACH

4

EVALUATION

5

CONCLUSION

❑ Why Logs Matter?

- ❑ **Logs are the first line of evidence for fault localization, performance diagnosis, and security monitoring.**



Fig. 1. Log (Digital footprint) | GIF [FIGMA]

But Real-World Logs Are:

- ❑ **Massive volume:** cloud platforms and HPC clusters emit logs at a very **large scale**.
- ❑ **Noisy and heterogeneous:** the same event appears in slightly **different formats** across services.
- ❑ **Formats constantly drift:** every service update can **silently change a template**.

The Average (Average of Individual System Velocities) is: $\approx 32,757$ lines per hour

Table1. Log Generation Velocity and Volume Benchmarks Across Distributed Systems

Software System	Annotated Logs	Collection Time (Time Span)
HDFS	11167740	38.7 Hours
Zookeeper	74273	26.7 Days
BGL	4631261	214.7 Days
Thunderbird	16601745	244 Days
Linux	23921	263.9 Days
Mac	100314	7.0 Days
Apache	51977	263.9 Days
OpenSSH	638946	28.4 Days
HealthApp	212394	10.5 Days
Average	3722508	2,828.6 Hours

Table source: <https://github.com/logpai/loghub-2.0>

The Real-Time Wall:

- ❑ Production systems emit tens of thousands of logs per second.
- ❑ Any method that keeps an **LLM on the live path** pays **latency and cost** that scale with log volume.

Table2. Live-Path LLM Latency Mitigation Frameworks

Paper	Core Live-Path Bottleneck	Key Architectural Mitigation
LogBatcher (Chiang et al., 2024)	Running the LLM for every individual log line creates massive delays and high token costs .	Caches similar logs and sends only unique items to the LLM in large batches.
LogParser-LLM (Zhong et al., 2024)	High semantic accuracy is heavily bottlenecked by frequent, sequential network and model inference execution.	Uses a fast tree structure for live streaming and keeps the LLM as a background fallback.
LLM meets ML / FlexLog (Hadadi et al., 2025)	Commercial closed-source APIs induce high financial overhead and highly unpredictable response times during live monitoring.	Pairs small local models with memory caches to guarantee sub-second processing delays.
LogPPT (Le & Zhang, 2023)	Raw streaming scales token count linearly , exhausting context windows and introducing network round-trip delays.	Extracts unique log patterns using quick mathematical filters before the LLM ever sees them.
LogGPT (Han et al., 2023)	Autoregressive, token-by-token sequence generation degrades runtime performance .	Runs a fast traditional parser to convert raw log text into short numerical IDs.

The Real-Time Wall:

- ❑ Rule-based systems are fast but break the moment **log formats change**.
- ❑ ML/DL models fail at high volumes due to **slow data prep** and heavy compute costs.

Table3. Live-Path Limitations of Traditional Log Analysis Paradigm

Paradigm	Representative Citations	Core Live-Path Strengths	Key Live-Path Bottleneck (Limitation)
Rule-Based Systems	He et al., 2017 (Drain); Du & Li, 2016 (Spell); Jiang et al., 2024	Super fast, have zero delays, and are completely easy to understand.	Require too much manual setup and fail when log formats change.
Machine Learning (ML)	Xu et al., 2009; Wang et al., 2004; Le & Zhang, 2021	Great at spotting basic patterns and number-based anomalies.	Need a data cleaning and manual prep to handle fast, live streams.
Deep Learning (DL)	Du et al., 2017 (DeepLog); Guo et al., 2021 (LogBERT); Fu et al., 2023 (Mlog); Wu & Kwon, 2024	Find complex, long-range text patterns without manual help.	They act like an unpredictable black box and cost too much computing power to run live.

Research Questions:

"To bridge these gaps, we investigate a core questions:"

- ❑ **RQ1 (Scalability):** Can one parser match LLM-level accuracy while keeping the runtime cost of a classical syntactic parser?
- ❑ **RQ2 (Reliability):** How stable does it stay under format drift, templates unseen at calibration time?
- ❑ **RQ3 (Real-time robustness):** Can a lightweight detector stay accurate on noisy, incomplete, reordered streams?

Our Contributions:

- ❑ **Compiled contextual log parser:** turns semantic templates into **executable functions for latency-bound** runtime parsing with NO online LLM inference.
- ❑ **Perturbation-aware anomaly detector:** a lightweight Transformer trained with noise injection for **stable detection** under corrupted, out-of-order logs.
- ❑ **Adaptive drift handling:** absorbs **unseen templates** by background re-compilation, with zero disruption to the live pipeline.
- ❑ **The shift:** **Zero** LLM calls on the live path.

PROPOSED APPROACH

Our Core Idea: Separation of Concerns

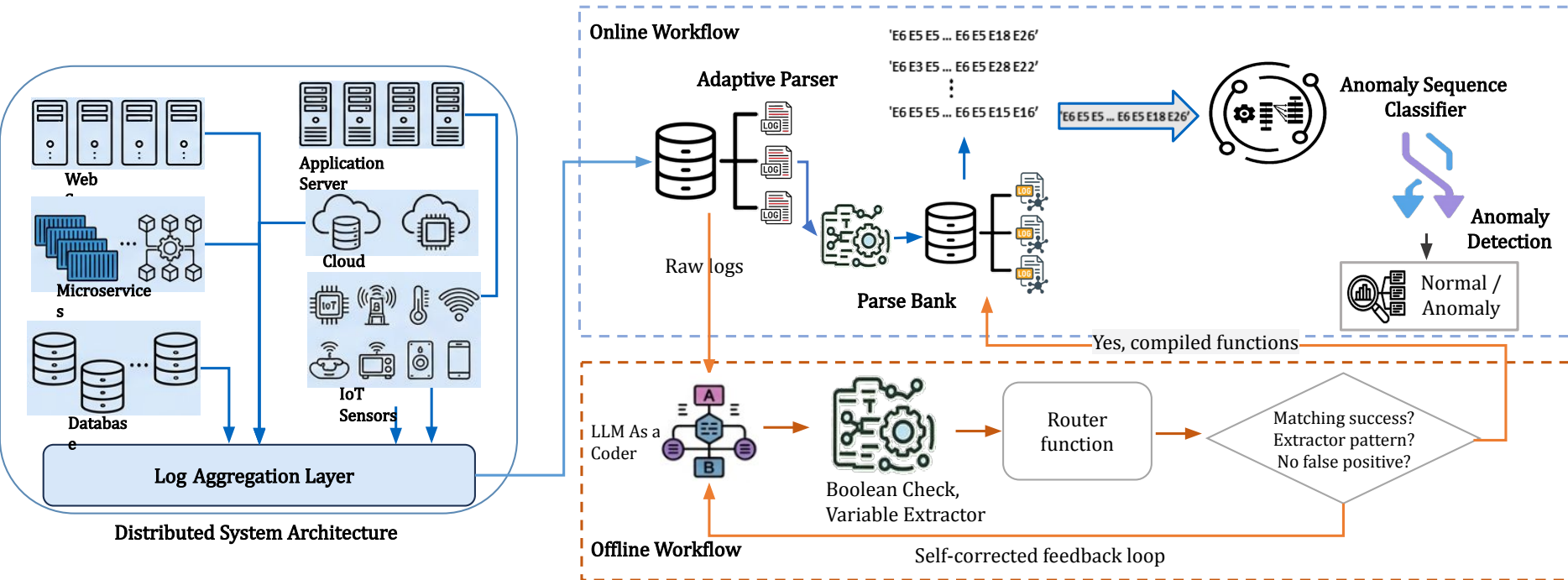


Fig. 2. CLAD Overall Architecture

PROPOSED APPROACH

CLAD PARSER:

- ❑ **Runs once** at startup, and again only when **new templates** appear
- ❑ Samples a **few representative log** lines per template
- ❑ LLM generates a **small parsing function** for each template
- ❑ **Self-validates** each **function** before accepting it
- ❑ Compiles validated functions into a **lightweight**, hot-reloadable **Parser Bank**

Handling Unseen Logs:

- ❑ Unmatched lines are labeled **UNKNOWN** and **buffered**
- ❑ Offline stage is **re-triggered** on the buffer to synthesize missing parsers
- ❑ New functions are hot-loaded, no runtime LLM

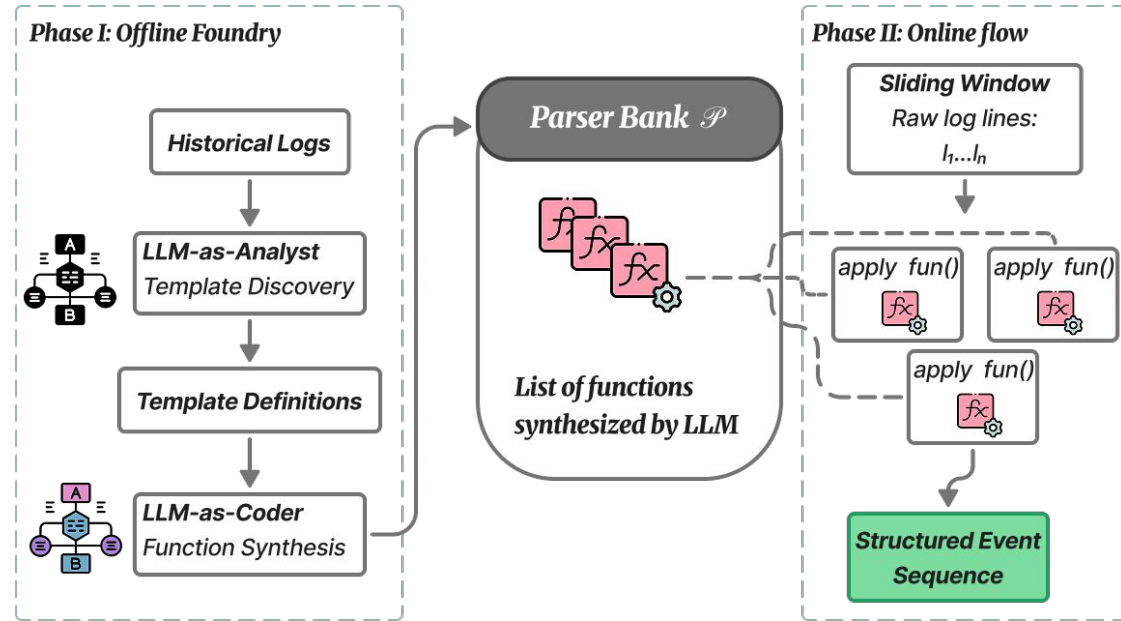


Fig. 3. Offline and Online phase of the parser Bank

PROPOSED APPROACH

Offline Phase:

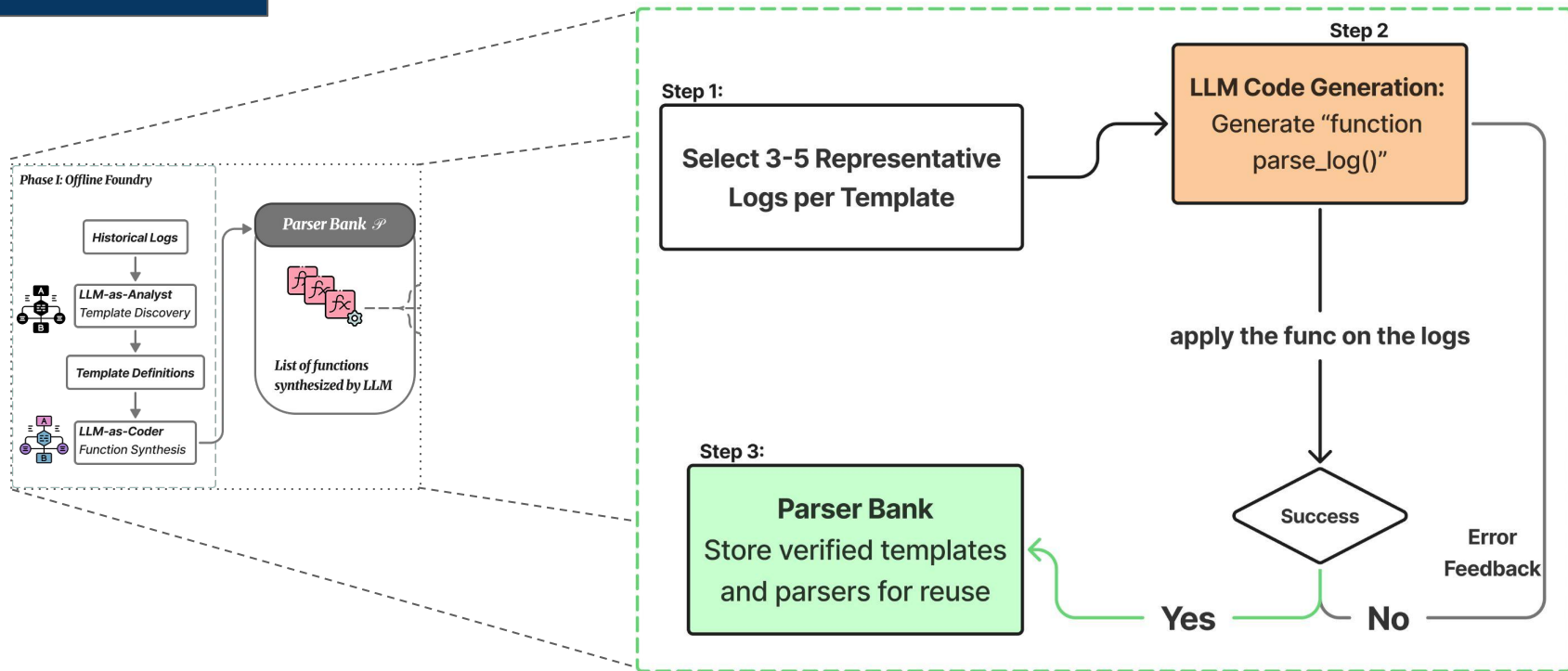


Fig. 4. Parse-rbank generation flowchart

PROPOSED APPROACH

Classifier:

Sequence Embedding:

- Each log event is converted into a dense vector and combined with positional encoding to preserve event order

Two-Layer Transformer Encoder:

- Stacked self-attention layers learn contextual relationships and long-range patterns across log sequences
- Motivation behind choosing only **Two-Layer** is
 - Lightweight**
 - We tried more layer but the two layer achieve the target accuracy**

Sequence Aggregation:

- Hidden states are pooled (mean) to form a single sequence representation.

Binary classification: Anomaly/Normal

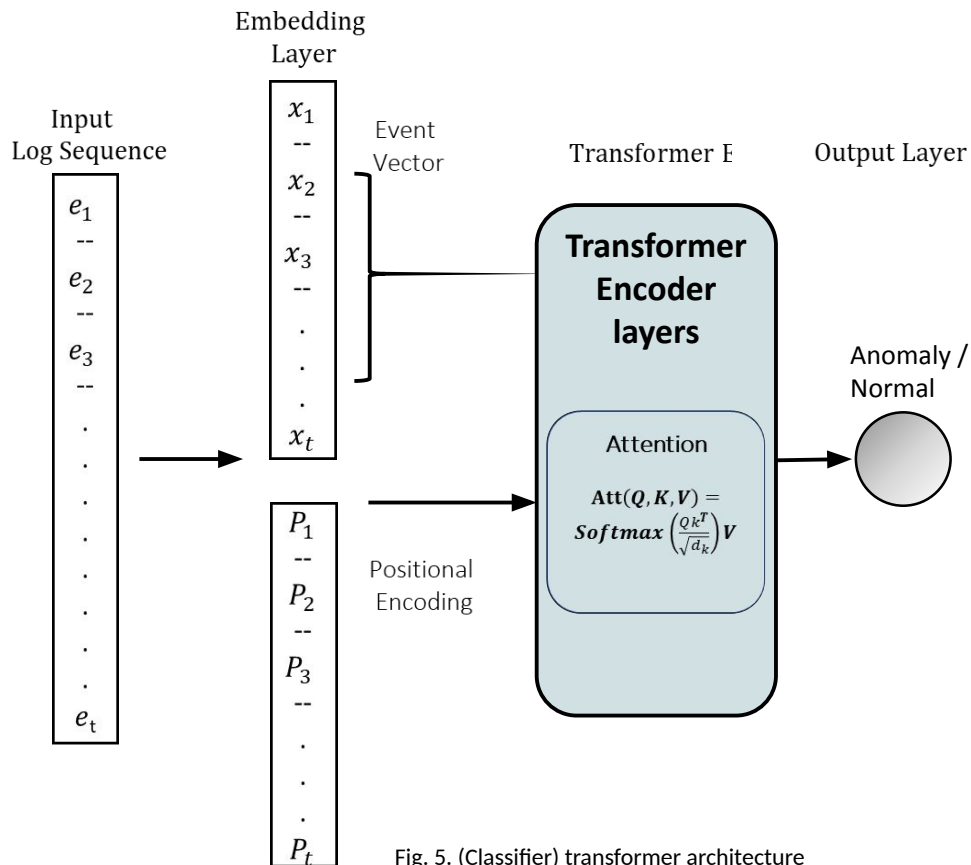


Fig. 5. (Classifier) transformer architecture

Evaluation structure:

- ❑ We organise the evaluation around three research questions that map directly to our contributions of this work.

RQ1 (Scalability)

Can one parser hit LLM-level accuracy at classical-parser cost?

RQ2 (Reliability)

How well does CLAD-Parser generalise to unseen and drifting log formats compared with both families?

RQ3 (Real-time robustness)

Can a lightweight detector stay accurate on noisy, out-of-order streams?

Evaluation structure:

- ❑ We organise the evaluation around three research questions that map directly to our contributions of this work.

RQ1 (Scalability)

Can one parser hit LLM-level accuracy at classical-parser cost?

RQ2 (Reliability)

How well does CLAD-Parser generalise to unseen and drifting log formats compared with both families?

RQ3 (Real-time robustness)

Can a lightweight detector stay accurate on noisy, out-of-order streams?

Evaluation structure:

- ❑ We organise the evaluation around three research questions that map directly to our contributions of this work.

RQ1 (Scalability)

Can one parser hit LLM-level accuracy at classical-parser cost?

RQ2 (Reliability)

How well does CLAD-Parser generalise to unseen and drifting log formats compared with both families?

RQ3 (Real-time robustness)

Can a lightweight detector stay accurate on noisy, out-of-order streams?

DATASET & SETUP

- ❑ **Test Environment:** AMD Ryzen Threadripper CPU (16 cores) | NVIDIA RTX 4090 GPU (23GB VRAM) | 64GB RAM.
- ❑ **Parser Performance Metrics**
 - ❑ **Grouping Accuracy (GA):** Measures if log messages are clustered into the exact right groups.
 - ❑ **Parsing Accuracy (PA):** The strictest line-level check; requires a perfect template match after variable removal.
 - ❑ **Template Accuracy (TA):** Checks the total template count to penalize over-splitting or over-merging.

Table 5. Statistics of the datasets used in this study

Dataset	Description	Time Span	#Log Messages	Anomaly Rate	Labeled	Purpose in Our Work
HDFS	Hadoop Distributed File System	38.7 hours	11.2M	2.93%	Yes	Classification & Parsing
BGL	Blue Gene/L Supercomputer (HPC)	7 months	4.7M	7.34%	Yes	Classification & Parsing
Thunderbird	Thunderbird Supercomputer (HPC)	244 days	211.2M	3.44%	Yes	Classification & Parsing
OpenStack	Cloud Infrastructure Log	-	207.8K	-	Yes	Parsing
Apache	Web Server Error Log	263.9 days	56.5K	-	No	Parsing
Android v1	Android Framework Log	-	1.5M	-	No	Parsing
HPC	High-Performance Cluster	-	433.5K	-	No	Parsing
Hadoop	Hadoop MapReduce Job Log	-	394.3K	-	Yes	Parsing
HealthApp	Health Application Log	10.5 days	253.4K	-	No	Parsing
Spark	Spark Job Log	-	33.2M	-	No	Parsing
Zookeeper	Zookeeper Service Log	26.7days	74.4K	-	No	Parsing

EVALUATION

DATASET & SETUP

- ❑ **Test Environment:** AMD Ryzen Threadripper CPU (16 cores) | NVIDIA RTX 4090 GPU (23GB VRAM) | 64GB RAM.
- ❑ **Reliability & Classifier Metrics:**
 - ❑ **Unseen Parsing Accuracy (UPA):** Tests parsing accuracy *only* on brand-new log formats never seen during offline calibration.
 - ❑ **Robustness Gap :** The difference between normal accuracy and unseen accuracy (PA-UPA). **A smaller gap means the system resists format drift.**
 - ❑ **Anomaly Classifier:** Evaluated using standard **Accuracy, Precision, Recall, and F1-Score.**

Table 5. Statistics of the datasets used in this study

Dataset	Description	Time Span	#Log Messages	Anomaly Rate	Labeled	Purpose in Our Work
HDFS	Hadoop Distributed File System	38.7 hours	11.2M	2.93%	Yes	Classification & Parsing
BGL	Blue Gene/L Supercomputer (HPC)	7 months	4.7M	7.34%	Yes	Classification & Parsing
Thunderbird	Thunderbird Supercomputer (HPC)	244 days	211.2M	3.44%	Yes	Classification & Parsing
OpenStack	Cloud Infrastructure Log	-	207.8K	-	Yes	Parsing
Apache	Web Server Error Log	263.9 days	56.5K	-	No	Parsing
Android v1	Android Framework Log	-	1.5M	-	No	Parsing
HPC	High-Performance Cluster	-	433.5K	-	No	Parsing
Hadoop	Hadoop MapReduce Job Log	-	394.3K	-	Yes	Parsing
HealthApp	Health Application Log	10.5 days	253.4K	-	No	Parsing
Spark	Spark Job Log	-	33.2M	-	No	Parsing
Zookeeper	Zookeeper Service Log	26.7days	74.4K	-	No	Parsing

RQ1: CLAD-Parser matches LLM-level accuracy while running at syntactic-parser speed and cost.

Scalability

- CLAD-Parser outperforms both syntactic and LLM-based baselines across all parsing metrics, especially on template accuracy and heterogeneous systems, by pairing a semantic offline stage with deterministic runtime parsing.

Table 6. Parsing Accuracy

Family	Parser	Android	Apache	BGL	HDFS	HPC	Hadoop	HealthApp	OpenStack	Spark	Thunderbird	Zookeeper	Avg
<i>(Parsing Accuracy)</i>													
<i>Classical</i>	Drain [4]	0.91	1.00	0.96	0.99	0.88	0.94	0.78	0.73	0.92	0.95	0.96	0.91
	IPLoM [15]	0.77	1.00	0.93	1.00	0.82	0.95	0.82	0.33	0.92	0.66	0.96	0.83
	LenMa [16]	0.87	1.00	0.68	0.99	0.82	0.88	0.17	0.33	0.88	0.94	0.84	0.76
	LFA [17]	0.61	1.00	0.85	0.88	0.81	0.90	0.54	0.20	0.99	0.64	0.83	0.75
	MoLFI [18]	0.77	1.00	0.94	0.99	0.81	0.65	0.31	0.21	0.40	0.65	0.83	0.69
	Spell [5]	0.58	0.57	0.26	0.31	0.62	0.62	0.63	0.14	0.91	0.84	0.91	0.58
	LKE [19]	0.40	1.00	0.17	0.84	0.41	0.48	–	0.73	0.63	0.55	0.71	0.59
	LogMine [20]	0.90	0.30	0.63	0.35	0.81	0.42	0.16	0.74	0.02	0.21	0.68	0.47
<i>Learn</i>	AEL [21]	0.19	0.29	0.42	0.93	0.00	0.08	0.01	0.07	0.36	0.01	0.38	0.25
	UniParser [32]	0.40	0.69	0.38	0.00	0.64	0.16	0.28	0.11	0.21	0.09	0.51	0.32
<i>LLM-based</i>	LogPPT [33]	0.46	0.99	0.77	0.00	0.81	0.17	0.65	0.41	0.65	0.40	0.66	0.54
	LogPrompt [22]	0.33	1.00	0.93	0.32	0.48	0.57	0.76	0.23	0.50	0.41	0.97	0.59
	LogParser-LLM [†] [26]	0.06	0.23	0.34	0.00	0.32	0.01	0.39	0.06	0.11	0.01	0.16	0.15
	LUNAR [25]	0.44	0.99	0.81	0.00	0.79	0.18	0.54	0.37	0.51	0.09	0.51	0.48
	LILAC [23]	0.79	1.00	0.88	0.00	0.80	0.15	0.75	0.36	0.78	0.38	0.37	0.57
	LogBatcher [24]	0.50	1.00	0.88	0.00	0.84	0.16	0.77	0.37	0.77	0.36	0.65	0.57
<i>Ours</i>	CLAD-Parser	0.99	1.00	0.98	1.00	0.96	0.98	0.98	0.89	0.98	0.99	0.99	0.98

EVALUATION

Scalability

- **CLAD-Parser** outperforms both syntactic and LLM-based baselines across all parsing metrics, especially on template accuracy and heterogeneous systems, by pairing a semantic offline stage with deterministic runtime parsing.

Table 6. Grouping Accuracy

(Grouping Accuracy)													
Classical	Drain [4]	0.91	1.00	0.96	1.00	0.89	0.95	0.78	0.73	0.92	0.95	0.97	0.91
	IPLoM [15]	0.77	1.00	0.94	1.00	0.82	0.95	0.83	0.33	0.92	0.66	0.96	0.84
	LenMa [16]	0.88	1.00	0.68	1.00	0.83	0.89	0.17	0.33	0.88	0.94	0.84	0.77
	LFA [17]	0.62	1.00	0.85	0.89	0.82	0.90	0.55	0.20	0.99	0.65	0.84	0.75
	MoLFI [18]	0.67	1.00	0.94	1.00	0.82	0.68	0.52	0.21	0.42	0.66	0.84	0.71
	Spell [5]	0.58	0.58	0.27	0.32	0.63	0.62	0.64	0.14	0.92	0.84	0.91	0.59
	LKE [19]	0.41	1.00	0.40	0.84	0.42	0.49	0.00	0.73	0.00	0.55	0.72	0.50
	LogMine [20]	0.90	0.31	0.64	0.35	0.81	0.42	0.16	0.74	0.03	0.22	0.69	0.48
	AEL [21]	0.19	0.29	0.43	0.93	0.00	0.08	0.01	0.08	0.36	0.02	0.39	0.25
Learn	UniParser [32]	0.78	1.00	0.95	1.00	1.00	0.81	0.74	0.53	0.93	0.96	0.99	0.88
	LogPPT [33]	0.85	1.00	0.98	1.00	0.90	0.97	1.00	1.00	1.00	0.98	1.00	0.97
LLM-based	LogPrompt [22]	0.10	0.00	0.48	0.00	0.23	0.13	0.21	0.00	0.03	0.08	0.35	0.15
	LogParser-LLM [†] [26]	0.47	0.30	0.18	0.00	0.64	0.49	0.33	0.06	0.00	0.12	0.08	0.24
	LUNAR [25]	0.83	1.00	0.99	1.00	0.95	0.98	0.78	0.49	1.00	0.96	0.99	0.91
	LILAC [23]	0.83	1.00	0.97	1.00	0.87	0.95	0.90	1.00	1.00	0.96	0.99	0.95
	LogBatcher [24]	0.74	1.00	0.99	1.00	0.95	0.99	0.92	0.49	1.00	0.95	0.99	0.91
Ours	CLAD-Parser	0.97	1.00	0.97	1.00	0.89	0.96	0.90	0.81	0.92	0.98	0.97	0.94

Scalability

- **CLAD-Parser** outperforms both syntactic and LLM-based baselines across all parsing metrics, especially on template accuracy and heterogeneous systems, by pairing a semantic offline stage with deterministic runtime parsing.

Table 6. Template Accuracy

<i>(Template Accuracy)</i>													
<i>Classical</i>	Drain [4]	0.67	0.50	0.20	0.29	0.37	0.26	0.28	0.07	0.33	0.28	0.34	0.33
	IPLoM [15]	0.20	0.50	0.13	0.00	0.39	0.18	0.19	0.00	0.25	0.18	0.24	0.21
	LenMa [16]	0.66	0.00	0.12	0.07	0.41	0.15	0.29	0.07	0.19	0.24	0.24	0.22
	LFA [17]	0.34	0.33	0.07	0.07	0.30	0.11	0.17	0.02	0.17	0.15	0.14	0.17
	MoLFI [18]	0.07	0.00	0.10	0.00	0.41	0.11	0.17	0.00	0.14	0.15	0.22	0.12
	Spell [5]	0.19	0.17	0.05	0.07	0.30	0.13	0.20	0.00	0.19	0.15	0.24	0.16
	LKE [19]	0.08	0.00	0.10	0.00	0.17	0.08	0.00	0.02	0.00	0.05	0.16	0.06
	LogMine [20]	0.34	0.00	0.12	0.00	0.41	0.18	0.32	0.02	0.19	0.28	0.24	0.19
	AEL [21]	0.10	0.00	0.00	0.14	0.02	0.01	0.03	0.05	0.03	0.00	0.08	0.04
<i>Learn</i>	UniParser [32]	0.46	0.50	0.31	0.29	0.50	0.54	0.47	0.21	0.50	0.50	0.64	0.45
	LogPPT [33]	0.47	0.83	0.79	0.21	0.74	0.76	0.91	0.95	0.94	0.75	0.72	0.74
<i>LLM-based</i>	LogPrompt [22]	0.54	1.00	0.54	0.00	0.57	0.30	0.65	0.81	0.64	0.54	0.46	0.55
	LogParser-LLM [†] [26]	0.07	0.17	0.05	0.07	0.43	0.13	0.17	0.12	0.31	0.13	0.18	0.17
	LUNAR [25]	0.51	0.83	0.75	0.29	0.76	0.51	0.67	0.81	0.61	0.65	0.56	0.63
	LILAC [23]	0.59	1.00	0.79	0.36	0.74	0.43	0.75	0.79	0.69	0.60	0.54	0.66
	LogBatcher [24]	0.41	1.00	0.82	0.36	0.78	0.57	0.87	0.74	0.58	0.68	0.62	0.68
<i>Ours</i>	CLAD-Parser	0.94	1.00	0.88	1.00	0.89	0.88	0.96	0.79	0.81	0.94	0.88	0.91

EVALUATION

Cost & Scalability

- CLAD-Parser** matches the speed and near-zero cost of syntactic parsers while achieving the accuracy of LLM-based methods, effectively removing the LLM from the online parsing path and combining strengths neither approach delivers alone.

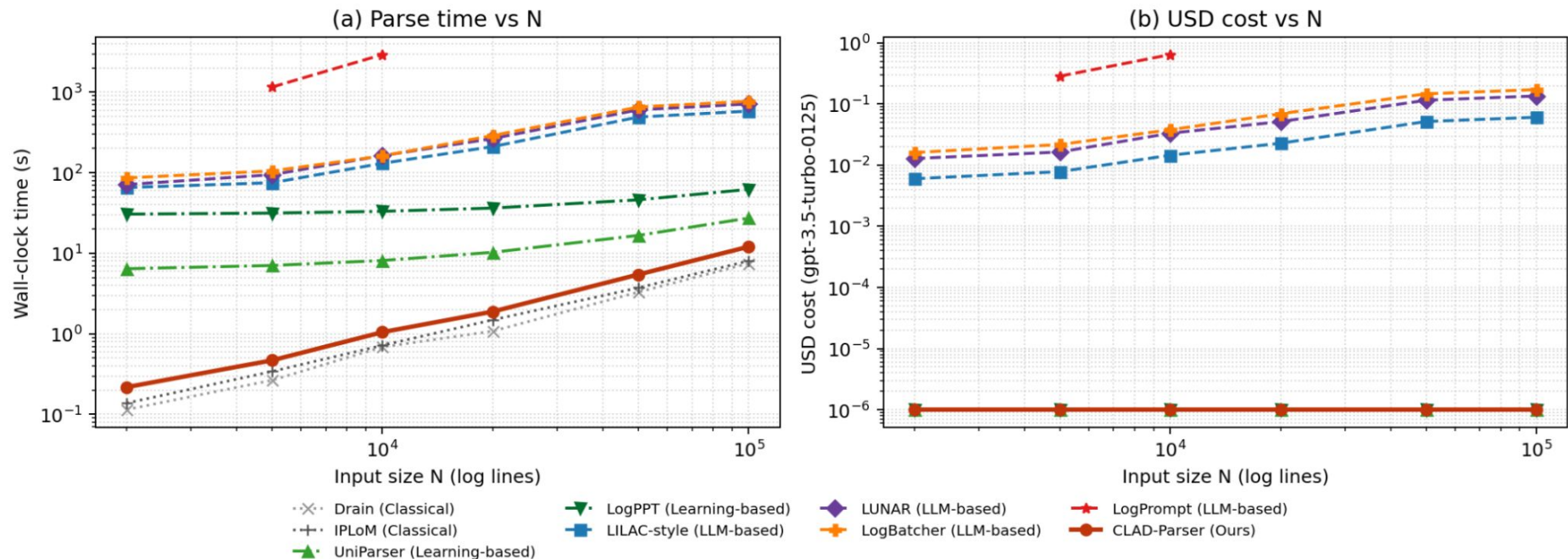


Fig 6. Scalability of CLAD-Parser against eight competing parsers on BGL and Thunderbird

RQ2: Robustness on Unseen and Drifting Logs

EVALUATION

Robustness on Unseen and Drifting Logs

- ❑ **CLAD-Parser** stays consistently accurate on unseen log formats, while syntactic parsers degrade sharply and LLM-based parsers grow unstable, because new templates are absorbed into its offline template bank without retraining or added runtime cost.

Robustness comparison: distribution of per-system metrics (narrower box + higher median = more robust & accurate)

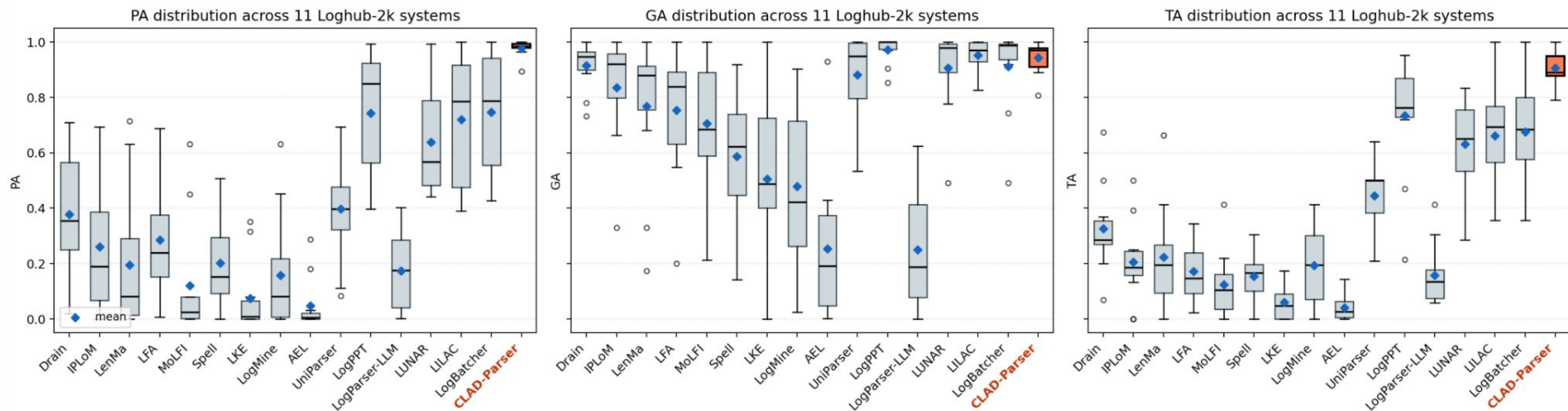


Fig 7. Parser robustness on unseen log templates.

RQ3: (Real-time robustness) Can a lightweight detector stay accurate on noisy, out-of-order streams?

EVALUATION

Classifier

- CLAD-Classifier** delivers strong, consistent anomaly detection across HDFS, BGL, and Thunderbird, outperforming recent sequential models and confirming that its transformer-based approach generalizes well across diverse log environments.

Table 7. Performance comparison of CLAD-Classifier with State-of-the-Art

Models	HDFS			BGL			Thunderbird		
	Pr(%)	Re(%)	F1(%)	Pr(%)	Re(%)	F1(%)	Pr(%)	Re(%)	F1(%)
LogRobust (Seq.)	0.93	0.95	0.94	0.93	0.90	0.90	0.81	0.87	0.79
Deeplog (Seq.)	0.91	0.78	0.84	0.78	0.76	0.77	0.45	0.61	0.52
LogBERT (Seq.)	0.86	0.78	0.82	0.87	0.81	0.84	0.89	0.92	0.91
LogLLM (Seq.)	0.99	1.00	0.99	0.86	0.97	0.91	0.96	0.96	0.96
LLM-LADE (Seq.)	0.98	0.99	0.99	0.99	0.98	0.98	0.99	0.99	0.99
LogSynergy (Seq.)	--	--	--	0.92	0.94	0.93	0.92	0.86	0.89
Pre-LogMGAE (Graph)	0.98	0.99	0.99	0.97	0.92	0.94	0.93	0.97	0.95
CLAD-Classifier (Seq.)	0.95	0.99	0.97	0.99	0.99	0.99	0.99	0.99	0.99

EVALUATION (Ablation Study)

Parser bank

- The ablation confirms the **Parser Bank** is essential, not just the LLM: relying on direct LLM inference alone is far too slow and less accurate, while adding the Parser Bank delivers both a large speedup and a substantial accuracy gain.

Table 8. Impact of Parser Bank on parsing performance and runtime

Approach	PA	GA	Time (s)
Qwen2.5-Coder-7B (Zero-Shot)	44.27	56.93	5,141
Qwen2.5-Coder-7B (Few-Shot)	58.27	65.18	5,924
Qwen2.5-Coder-7B + Parser Bank	93.00	96.00	169

Classifier

- **CLAD-Classifier** stays far more stable under increasing **log noise** than **LSTM** and **BERT**, thanks to its contextual transformer modeling and perturbation-aware training, allowing it to maintain strong detection.

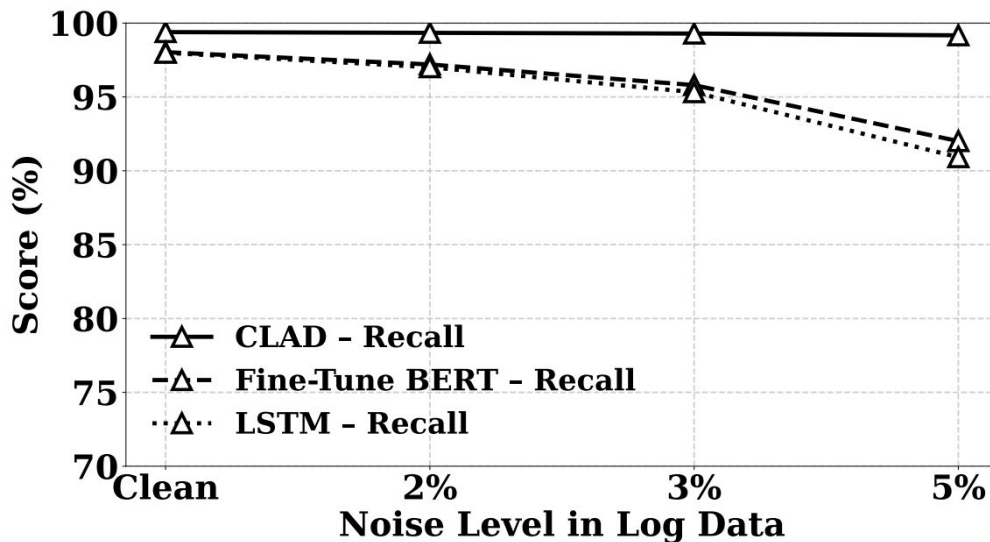


Fig 8. Recall Comparison of CLAD-Classifier

EVALUATION (Ablation Study)

Simulation of CLAD Online Workflow

- CLAD's simulated **online workflow** achieves **high throughput** and **sub-millisecond latency** on both datasets without sacrificing detection **accuracy**, confirming it's practical for real-time production deployment.

Table 9. Test results of the CLAD online workflow

Dataset	Throughput (logs/s)	Latency (ms)	Prec.	Rec.	F1
BGL	43174	0.028	0.9906	0.9765	0.9835
Thunderbird	1162	0.861	0.9986	0.9923	0.9954

KEY Findings

- ❑ **SOTA** accuracy in parsing and anomaly detection
- ❑ **Real-time**, sub-millisecond processing sustained end-to-end
- ❑ Closes the accuracy-vs-speed trade-off of prior **LLM-based** methods

Limitations

- ❑ Evaluated on a single node only, no geo-distributed testing
- ❑ Format drift needs periodic offline recalibration, not full auto-adaptation

Future Work

- ❑ Extend fault tolerance to geo-distributed settings
- ❑ Integrate with automated recovery protocols
- ❑ Explore extending CLAD toward fault localization and root cause analysis
- ❑ Goal: strengthen overall system availability



Thank you

Q & A

Contact me

